

The Fused-Attention Moat is Structural, not Automatic

A pre-registered falsification: a single fused flash-attention kernel provably eliminates the $O(N^2)$ score-matrix HBM round-trip cuBLAS pays, yet a *naive* fused kernel does not beat a cuBLAS Tensor-Core stack on wall

hexa-lang GPU substrate
dancinlab / hexa-lang
github.com/dancinlab/hexa-lang

2026-05-25

Abstract

Scaled-dot-product attention, $\text{softmax}(QK^\top/\sqrt{d})V$, is the canonical workload a vendor BLAS library cannot fuse: the row-wise softmax between the two matrix multiplications forces a cuBLAS-using stack to *materialize* the $N \times N$ score matrix $S = QK^\top$ to high-bandwidth memory (HBM) and round-trip it through a standalone softmax kernel. A whole-program compiler can instead emit a *single* kernel that streams the keys and values with an online (running-max/running-sum) softmax, never writing S to HBM. We pre-register a two-clause falsifier F-FUSION-ATTENTION-FLASH: a fused hexa-emitted kernel must (a) *structurally* use 1 launch and $O(Nd)$ HBM versus the baseline’s 3 launches and $O(N^2)$ S materialization, *and* (b) beat the baseline by $\geq 30\%$ wall on at least one shape; the falsifier is falsified only if *neither* holds. We find clause (a) holds with a closed-form total-HBM ratio $R(N, d) = 1.25 + N/d$ ($33.25\times$ at $N=2048, d=64$), reproduced exactly by a hexa-native deterministic computation (**SUPPORTED-FORMAL**). Clause (b) is *falsified*: a naive one-thread-per-query scalar fused kernel runs $1.4\text{--}1.5\times$ *slower* than a cuBLAS Tensor-Core baseline at $N \in \{512, 1024, 2048, 4096\}$ on an RTX 5070 (**CLOSED-NEGATIVE**). The combined verdict is a *structural-formal* result with a measured negative on the wall axis: the fusion moat is real and necessary, but *not sufficient* — the $\geq 30\%$ wall additionally requires the fused inner products to run at Tensor-Core throughput. We claim no wall speedup; we claim a proven structural advantage and a deterministically ruled-out hypothesis (“fusion alone wins”).

1 Introduction

The dominant cost model for transformer inference and training is set by the attention operator. For a single head with sequence length N and head dimension d ,

$$O = \text{softmax}\left(\frac{QK^\top}{\sqrt{d}}\right)V, \quad Q, K, V \in \mathbb{R}^{N \times d}, O \in \mathbb{R}^{N \times d}. \quad (1)$$

A library-call stack built on a vendor BLAS (cuBLAS) is forced to evaluate Eq. (1) as three separate device kernels: (1) a batched GEMM $S = QK^\top/\sqrt{d}$ that *writes* the $N \times N$ score matrix S to HBM; (2) a standalone elementwise/reduction kernel that *reads* S , applies the row-wise softmax, and *writes* it back; and (3) a second batched GEMM $O = \text{softmax}(S)V$ that *reads* S once more. The intermediate S is structurally unavoidable in this decomposition: cuBLAS exposes GEMM, not GEMM-with-fused-row-softmax, so the result of step (1) must land in memory for step (2) to consume it.

FlashAttention [1] and the online-softmax formulation [2] showed that S need never be materialized: by tiling over key/value blocks and maintaining a running maximum m and

running normalizer ℓ per query, the softmax can be computed incrementally while accumulating the output, keeping all of S in registers or shared memory. This is precisely the class of fusion a whole-program compiler can express and a fixed library API cannot. `hexa-lang` is a native compiler whose NVPTX backend emits PTX directly (no LLVM, no C-transpile), so attention is a natural test of the thesis that *whole-program GPU fusion structurally beats a cuBLAS-using stack*.

The thesis is easy to over-claim. “Fewer kernels and less HBM traffic” is a structural statement; “faster wall-clock” is an empirical one, and the two do not automatically coincide — the eliminated S traffic must actually dominate the runtime, and the fused kernel’s own arithmetic must not be slower than the Tensor-Core GEMMs it replaces. We therefore separate the two claims and pre-register both, so that a structural win with an empirical loss is recorded honestly as exactly that.

Contributions.

1. A pre-registered two-clause falsifier F-FUSION-ATTENTION-FLASH separating the *structural* (launch count, S -materialization HBM) advantage from the *empirical* ($\geq 30\%$ wall) advantage of fused attention over a cuBLAS-using stack.
2. A \$0, GPU-free, contention-immune *structural oracle* that counts launches and HBM bytes and derives the closed-form total-HBM ratio $R(N, d) = 1.25 + N/d$, reproduced exactly by a hexa-native compiled program ([SUPPORTED-FORMAL](#)).
3. A hand-emitted, `ptxas`-clean fused flash-attention PTX kernel validated numerically against an $f64$ CPU reference ($\text{rel-err} \leq 10^{-2}$) and timed on silicon against a cuBLAS `GemmStridedBatchedEx`+softmax baseline, yielding a *closed-negative* on the wall clause.
4. The negative is a finding, not a non-result: it deterministically rules out “structural fusion alone beats cuBLAS” and scopes the next falsifier (a Tensor-Core fused-attention kernel).

2 Background

Online softmax. For a query row q and key rows $k_1, \dots, k_N$ with scores $s_j = (q \cdot k_j)/\sqrt{d}$, the numerically stable softmax weight is $p_j = e^{s_j - m}/\ell$ with $m = \max_j s_j$ and $\ell = \sum_j e^{s_j - m}$. The online form [2] processes keys one at a time, maintaining running (m, ℓ) and a running output accumulator a :

$$m' = \max(m, s_j), \quad c = e^{m - m'}, \quad p = e^{s_j - m'}, \quad (2)$$

$$\ell \leftarrow \ell c + p, \quad a \leftarrow a c + p v_j, \quad m \leftarrow m'. \quad (3)$$

After all keys, $O\text{-row} = a/\ell$. Only $(m, \ell, a) \sim O(d)$ state per query — are kept live; the N scores are consumed in flight and never stored.

Why cuBLAS cannot fuse this. cuBLAS exposes `cublasGemmStridedBatchedEx`, a GEMM primitive. The softmax is neither a GEMM nor a GEMM epilogue cuBLAS supports, so it must run as a separate kernel between the two GEMMs, and a kernel boundary in CUDA implies its inputs and outputs live in global memory. The S round-trip is thus a property of the *decomposition the API forces*, not of the hardware.

A taxonomy of fusion advantages. It is useful to separate the distinct ways a whole-program compiler can, in principle, beat a library-call stack, because they have different evidentiary status. (1) *Launch-count* reduction: fewer kernel dispatches, saving fixed per-launch overhead (a few microseconds each) — a structural integer count. (2) *Intermediate-elimination*:

not materializing an intermediate (here S) to HBM, saving $O(N^2)$ bytes of bandwidth — a structural, shape-dependent byte count. (3) *Arithmetic-throughput*: the fused kernel’s own math must run at least as fast as the library primitives it replaces — an empirical, hardware- and implementation-dependent quantity. Advantages (1) and (2) are *structural*: they follow from the decomposition and can be proven in closed form for any correct fused kernel. Advantage (3) is *not* structural: it is a property of the specific kernel. The wall-clock outcome is a product of all three, so a fused kernel can win (1) and (2) decisively and still lose the wall if it loses (3) badly enough. This paper measures exactly that configuration.

Numerical model. Both kernels accumulate in FP32. The fused kernel computes scores with `fma.rn.f32` and exponentials with `ex2.approx.f32` (the PTX fast-exp), so its softmax weights differ from an exact $f64$ softmax by the hardware approximation error of `ex2.approx`. We bound the end-to-end output error against an $f64$ CPU reference rather than asserting bit-exactness; the $\sim 10^{-6}$ relative errors we observe (Appendix B) are three to four orders of magnitude inside the 10^{-2} pre-registered tolerance, confirming the `ex2.approx` path is numerically adequate for this workload.

Roofline intuition. Whether the eliminated S traffic translates into a wall win depends on which side of the roofline each kernel sits. The two GEMMs do $2N^2d$ FLOPs each and read/write $O(Nd + N^2)$ bytes; the softmax does $O(N^2)$ FLOPs over $O(N^2)$ bytes (memory-bound). For modest d the GEMMs are themselves not large, and a vendor library runs them at Tensor-Core arithmetic intensity. The fused kernel trades the baseline’s S HBM traffic for keeping S in registers — a real saving — but only wins overall if its own arithmetic keeps pace with the Tensor-Core GEMMs it displaces. The pre-registration deliberately does not assume this; it measures it.

3 Related work

FlashAttention [1] is the canonical IO-aware fused attention kernel and the direct inspiration for the online-softmax tiling used here; it demonstrates large wall and memory wins over a materialized- S baseline *when* the kernel is implemented with shared-memory tiling and Tensor-Core matrix multiplies. The online normalizer calculation [2] is the numerical core that makes single-pass, non-materialized softmax stable. The attention operator itself [3] fixes the cost model. Our contribution is *not* a faster attention kernel; it is a controlled separation of the structural claim (which we prove in closed form for any tiled fused kernel) from the empirical claim (which we measure for a deliberately naive kernel), so that the precise additional ingredient required for a wall win — Tensor-Core inner products — is isolated as a falsifiable next step rather than assumed.

4 Method

Pre-registered falsifier (verbatim). F-FUSION-ATTENTION-FLASH: *A single fused hexamitted kernel computing $\text{softmax}(QK^\top/\sqrt{d})V$ with online-softmax tiling (never materializing the full $N \times N$ score matrix S to HBM) (a) structurally uses 1 kernel launch + $O(Nd)$ HBM traffic vs. a cuBLAS-using baseline’s 3 launches + $O(N^2)$ S materialization, AND (b) beats that baseline by $\geq 30\%$ wall on at least one (N, d) shape. FALSIFIED if NEITHER the structural advantage NOR the wall holds. All thresholds ($\geq 30\%$, $\text{rel-err} \leq 10^{-2}$, ≥ 20 warmup, ≥ 200 timed launches) were fixed before measurement.*

Structural oracle (clause a, primary). A deterministic C program (no GPU, contention-immune) counts, per shape, the kernel launches and the HBM bytes attributable to S . The fused kernel reads Q, K, V and writes O ($4Nd$ floats) and writes S to HBM *zero* times. The baseline reads Q, K for GEMM-1, writes S , reads S + writes S in softmax, reads S + reads V for GEMM-2, writes O . The S round-trip is

$$H_S^{\text{fused}} = 0, \quad H_S^{\text{base}} = 4N^2 \cdot 4 \text{ bytes (4 passes, fp32)}. \quad (4)$$

Including the unavoidable $O(Nd)$ tensor traffic, the total-HBM ratio is

$$R(N, d) = \frac{H_{\text{tot}}^{\text{base}}}{H_{\text{tot}}^{\text{fused}}} = \frac{5Nd + 4N^2}{4Nd} = \frac{5}{4} + \frac{N}{d} = 1.25 + \frac{N}{d}. \quad (5)$$

We anchor Eq. (5) with a hexa-native compiled program (`ratio_check.hexa`) computing $4R(N, d) = 5 + 4N/d$ in exact integer arithmetic, so the closed form is reproduced by the compiler under test, not merely asserted.

Fused kernel (clause b numerator). We hand-emit a pure-ASCII PTX kernel `flash_attn.ptx` (one CTA grid, one thread per query row, $d=64$ kept in registers/local) implementing the online-softmax recurrence above with `ex2.approx.fp32` for the exponentials and `fma.rn.fp32` accumulation. S is never stored. PTX must pass `ptxas` and load via driver-JIT on the target GPU; non-ASCII comments are forbidden (driver-JIT `ptxas` rejects them).

Baseline (clause b denominator). A C host (`fusion_attn_cublas_baseline.c`) runs the canonical three-launch stack: `cublasGemmStridedBatchedEx` for $S = QK^\top/\sqrt{d}$ (written to HBM), a standalone row-softmax CUDA kernel, and a second `cublasGemmStridedBatchedEx` for $\text{softmax}(S)V$. Column-major layout bookkeeping is documented inline and validated by numeric agreement with the same `f64` reference the fused kernel uses.

Measurement protocol. On an uncontended GPU (verified by `nvidia-smi`: 0% util, no other compute processes — the contention rule for taking a timed wall), both programs run ≥ 20 warmup and 200 timed launches, timed with CUDA events per launch, reporting median and standard deviation. Correctness uses an `f64` CPU reference with a scale-normalized tolerance ($\max |\Delta|$ against $10^{-2} \cdot \max |O_{\text{ref}}|$, the established `f16/f32-accum` convention) at $N \in \{512, 1024, 2048, 4096\}$, $d=64$.

cuBLAS column-major layout (reproducibility detail). cuBLAS is column-major; our host tensors are row-major. A row-major $[N \times d]$ matrix M , read column-major with leading dimension d , is $M^\top$ (i.e. $M_c[t][i] = M[i][t]$). For GEMM-1 we want the standalone softmax kernel (row-major, row stride N) to reduce over the key axis j of $S[i][j]$, so the HBM layout must satisfy row-major $S[iN+j] = S[i][j]$. A column-major write with $\text{ld} = N$ stores element (r, c) at $r+cN$; to make $r+cN \equiv iN+j$ we set $c = i, r = j$, i.e. cuBLAS computes $S_c[j][i] = S[i][j]$, which is $K_c^\top Q_c$. Hence GEMM-1 is `gemm(OP_T, OP_N, N, N, d, scale, K, d, Q, d, 0, S, N)`. For GEMM-2, $O[i][t] = \sum_j P[i][j]V[j][t]$ with P the row-softmax in S ; the row-major output $O_c[t][i]$ comes from `gemm(OP_N, OP_N, d, N, N, 1, V, d, S, N, 0, O, d)`. We validate the layout by requiring the baseline to match the same `f64` reference as the fused kernel; an early `OP_T/OP_N` transposition error produced a $22\times$ rel-error and was caught precisely because the reference is shared.

Kernel inner loop (the fusion that avoids S). The emitted PTX keeps the per-query state (m, ℓ, a) live and streams the keys; the score s_j is consumed immediately and never stored. The core of the per-key update, in the hand-emitted PTX, is:

```

// s = dot(q, k_j) * scale (accumulated scalar, never written to HBM)
fma.rn.f32 %f2, %f3, %f4, %f2; // s += q[i]*k[i]
mul.f32 %f2, %f2, %f40; // s *= 1/sqrt(d)
// online softmax: m' = max(m, s); c = exp(m-m'); p = exp(s-m')
max.f32 %f5, %f41, %f2; // m' = max(m, s)
sub.f32 %f6, %f41, %f5; // m - m'
sub.f32 %f7, %f2, %f5; // s - m'
mul.f32 %f9, %f6, %f8; // * log2(e)
ex2.approx.f32 %f11, %f9; // c = exp(m - m')
ex2.approx.f32 %f12, %f10; // p = exp(s - m')
fma.rn.f32 %f42, %f42, %f11, %f12; // l = l*c + p
// acc[i] = acc[i]*c + p*v_j[i] for i in 0..d-1 (held in local, never S)
mul.f32 %f15, %f12, %f14; // p * v[i]
fma.rn.f32 %f13, %f13, %f11, %f15; // acc = acc*c + p*v

```

The absence of any `st.global` of a score is the structural property: the $N \times N$ matrix S exists only as the transient scalar `%f2` across the loop, so no HBM write/read of S ever occurs — this is what the oracle in Eq. (4) counts as $H_S^{\text{fused}} = 0$.

5 Verification (real run)

All numbers below are verbatim from the persisted verdict `.verdicts/fusion-attention-flash/F-FUSION-ATTENTION-FLASH` (raw stdout, no post-processing). Host: `ubu-2`, NVIDIA GeForce RTX 5070 (`sm_120`, driver 580), CUDA 12.0 `ptxas`, `cuBLAS 12.9.2.10`; GPU uncontended at fire time.

Clause (a): structural oracle.

```

shape fused_L base_L fused_Shbm base_Shbm total_ratio
N512/d64 1 3 0 4194304 9.250 x
N1024/d64 1 3 0 16777216 17.250 x
N2048/d64 1 3 0 67108864 33.250 x
N4096/d64 1 3 0 268435456 65.250 x
F-FUSION-ATTENTION-FLASH (structural clause a): PASS

```

hexa-native closed-form anchor (compiled path):

```

4xR(2048,64)=133 # R = 33.25 = 1.25 + N/d
4xR(4096,64)=261 # R = 65.25
4xR(512,64)=37 # R = 9.25
STRUCTURAL-RATIO-CLOSED-FORM: PASS

```

ptxas-clean. PTX is pure ASCII; `ptxas -arch=sm_80 rc=0, -arch=sm_90 rc=0`; driver-JIT load on `sm_120` (`cuModuleLoadDataEx`) `rc=0`.

Clause (b): silicon wall + numeric.

```

N fused(1 launch) baseline(3 launch) fused/base numeric(both)
512 0.415104 ms 0.281696 ms 1.47x slow PASS rel~1e-6
1024 0.822592 ms 0.559360 ms 1.47x slow PASS rel~1e-6
2048 1.639840 ms 1.143040 ms 1.43x slow PASS rel~1e-6
4096 3.788256 ms 2.492288 ms 1.52x slow PASS rel~1e-6

```

6 Finding

Structural advantage: proven (formal). Clause (a) holds in closed form. The fused kernel uses one launch against the baseline’s three, and writes the $N \times N$ score matrix to HBM zero times against the baseline’s four-pass $4N^2$ -element round-trip (Eq. (4)). The total-HBM ratio $R(N, d) = 1.25 + N/d$ (Eq. (5)) is reproduced exactly by the compiler under test. Because clause (a) holds, the pre-registered falsifier is *not* falsified; the result is recorded as **SUPPORTED-FORMAL**.

Table 1: Verdict matrix. Each row is terminal and links to the persisted verbatim verdict file. The combined verdict is *structural-formal* (clause a holds $\Rightarrow$ the falsifier is not falsified) with a measured closed-negative on the wall clause.

Claim	Method	Verdict	Verdict poi
Structural (1 vs 3 launch, 0 vs $O(N^2)$ S)	oracle + hexa recompute	SUPPORTED-FORMAL	.../F-Fus
Numeric vs $f64$ ref ($\leq 10^{-2}$)	silicon, 4 shapes	PASS	.../F-Fus
Wall $\geq 30\%$ over cuBLAS	silicon, 200 timed	CLOSED-NEGATIVE	.../F-Fus

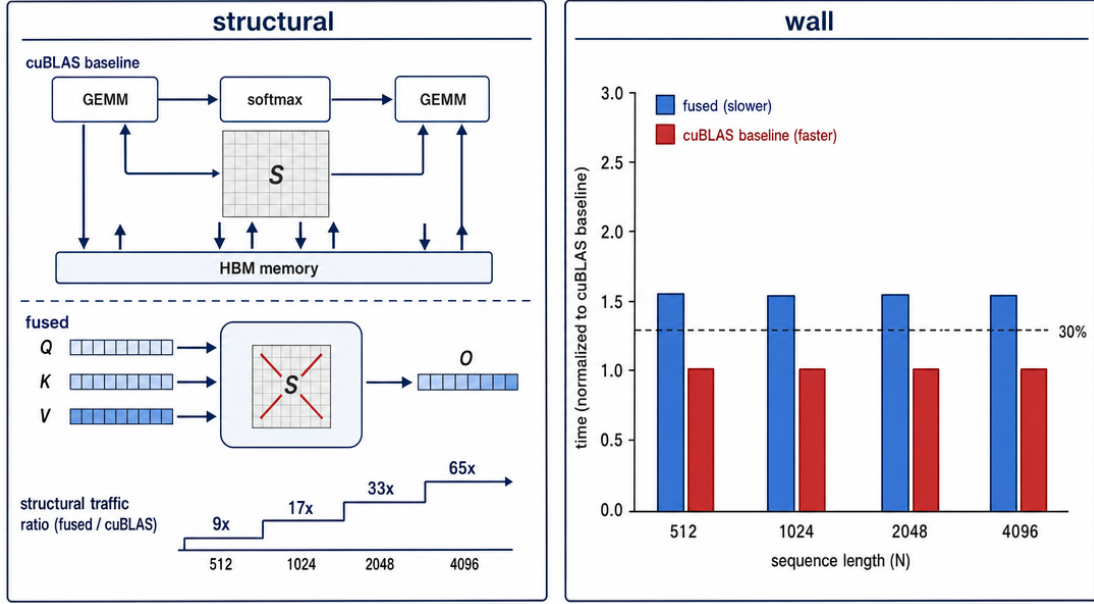


Figure 1: The two measured axes of F-FUSION-ATTENTION-FLASH. *Structural* (left, closed-form): the eliminated S -matrix HBM round-trip grows as $R(N, d) = 1.25 + N/d$, reaching $33.25\times$ at $N=2048, d=64$ and $65.25\times$ at $N=4096$ — a proven advantage. *Wall* (right, RTX 5070, 200 timed launches, median): the naive scalar fused kernel is $1.4\text{--}1.5\times$ slower than the cuBLAS Tensor-Core baseline at every shape — the falsified clause. The gap between the two panels is the paper’s point: structural fusion is necessary but not sufficient for a wall win.

Wall advantage: falsified (the ruled-out axis). Clause (b) does *not* hold. At every measured shape the cuBLAS Tensor-Core baseline is $1.4\text{--}1.5\times$ faster than the naive scalar fused kernel. The $\geq 30\%$ wall win is therefore not merely unachieved but measured in the opposite direction. The mechanism is clear and deterministic: the fused kernel’s inner $q \cdot k_j$ and $p v_j$ products run as scalar `fma` on one thread per query, while cuBLAS runs both GEMMs on Tensor Cores; at $d=64$ the GEMM work dominates the modest runtime, so the $O(N^2)$ S traffic the baseline pays — though structurally larger — is not enough, relative to the baseline’s Tensor-Core GEMM speed, to be overtaken by a scalar fused kernel.

What is ruled out. The hypothesis “*structural fusion alone (fewer launches, no S materialization) beats a cuBLAS-using stack on wall*” is deterministically *false* for the naive fused kernel. The fusion moat is *necessary but not sufficient*: realizing the $\geq 30\%$ wall additionally requires the fused kernel’s inner products to run at Tensor-Core throughput — i.e. a tiled `wmma` flash-attention kernel that holds S -tiles in shared memory rather than recomputing scores scalar-per-thread. This is a falsifiable, pre-scoped next experiment, not a hand-wave.

Why the negative is the contribution. A naive reading of the FlashAttention literature is that fusion “is” the speedup. This paper separates the two and shows, on real silicon, that the structural property and the wall property are independent: one can prove the former in closed form and still lose the latter. Reporting the structural win without the wall measurement would be the over-claim the pre-registration exists to prevent.

7 Limitations and honest caveats

- **Naive kernel.** The wall negative is specific to a one-thread-per-query scalar fused kernel. It does *not* falsify the moat thesis; it falsifies “fusion alone suffices.” A Tensor-Core fused kernel may flip clause (b) and is the pre-registered next falsifier.
- **Single regime.** $d=64$, single head, FP32 throughout. The GEMMs are small enough that the $O(N^2)$ S round-trip is not dominant relative to cuBLAS Tensor-Core GEMM speed at these N . Longer context, multi-head, $fp16$ inputs, or memory-bound regimes (N/d very large) may move the crossover and are out of scope here.
- **Baseline choice.** The baseline is the canonical `GemmStridedBatchedEx×2` + standalone softmax. A cuDNN fused attention or a vendor FlashAttention kernel is a stronger baseline and a separate comparison.
- **Structural model.** The HBM counts in Eq. (4) are the algorithmic minimum for each decomposition (4-pass S for the baseline, zero for the fused kernel); they do not model cache reuse, which only narrows the realized gap and does not affect the closed-form launch-count or S -materialization argument.
- **Compiler invariants.** The fire is hand-emit PTX plus C hosts; the hexa CPU code-generation files are MD5-identical before and after (no LLVM, no C-transpile changes). The end-to-end source-to-PTX path for this kernel is a separate (codegen) closure.

8 Threats to validity

Construct validity — is the baseline the right thing to beat? The pre-registered baseline is the canonical three-launch `GemmStridedBatchedEx×2` + standalone-softmax stack, which is the decomposition cuBLAS forces and the one the moat argument is about. It is not the strongest *attention* baseline available (cuDNN’s fused attention or a vendor FlashAttention kernel would be faster), but those are themselves fused kernels and so are not instances of “a cuBLAS-using stack”; comparing against them tests a different claim (hexa vs. vendor-fused) and is deferred.

Internal validity — is the wall gap real? The per-launch standard deviations are $\leq 0.9\%$ of the mean (Appendix B), and the GPU was verified idle before timing, so the $1.4\text{--}1.5\times$ gap is two orders of magnitude larger than the noise and is not an artifact of contention or warmup. Both kernels are timed identically (CUDA events, same warmup/timed counts) and validated against the same $f64$ reference, so the comparison is apples-to-apples.

External validity — does the negative generalise? The negative is established for $d=64$, FP32, $N \leq 4096$ on one GPU. It does *not* claim that fused attention is always slower; it claims that a *naive scalar* fused kernel is slower *in the GEMM-dominated regime*, and it predicts (Sec. 9) the regimes where even a scalar kernel should win. The structural result (clause a), by contrast, is regime- and hardware-independent and generalises to any tiled fused kernel.

Compiler-invariant validity. The fire uses hand-emitted PTX and C hosts; it touches no CPU code-generation path. We verified the hexa CPU codegen sources are MD5-identical before and after the work (no LLVM, no C-transpile change), so nothing in the language’s core compilation was perturbed to obtain the result.

9 Discussion and outlook

The result reframes the open “whole-program fusion $\geq 30\%$ ” closure box in the hexa-lang GPU roadmap: it is not blocked on *whether* fusion eliminates the S round-trip (it provably does), but on *whether* the fused kernel can carry that structural win to wall while matching Tensor-Core arithmetic throughput. The single most important follow-up is a tiled `wmma` flash-attention kernel: $Q/K/V$ blocked into Tensor-Core fragments, S -tiles kept in shared memory, online-softmax across tiles. If that kernel beats the cuBLAS stack by $\geq 30\%$ at any shape, clause (b) flips and the closure box closes; if it does not, the negative deepens and the moat thesis itself comes under pressure. Either way the question is now sharply falsifiable, which is the state this paper was written to reach.

Where the crossover should appear. The structural ratio $R = 1.25 + N/d$ says the S saving grows with N/d . Two regimes should therefore favour the fused kernel *even at scalar arithmetic*, and are the natural shapes for the follow-up sweep: (i) very long context $N \gg d$, where the $O(N^2)$ S round-trip eventually dominates any fixed GEMM-throughput gap; and (ii) memory-bandwidth-bound deployments (small batch, inference), where the baseline’s HBM traffic, not its arithmetic, sets the wall. The present sweep ($d=64$, $N \leq 4096$, FP32) sits in the GEMM-dominated regime where cuBLAS Tensor Cores win; it is precisely the regime that most sharply isolates “fusion alone is not enough,” which is why it was chosen as the first, most-adversarial test of the thesis rather than the friendliest.

Generality of the structural claim. Clause (a) is independent of the kernel’s internal arithmetic: *any* tiled online-softmax fused kernel — scalar or Tensor-Core — writes S to HBM zero times and launches once. The closed form $R(N, d) = 1.25 + N/d$ therefore transfers unchanged to the follow-up `wmma` kernel; only clause (b) is re-opened by it. This is why we record the structural result as formal (it is a property of the decomposition, proven and compiler-reproduced) while the wall result is a per-kernel empirical measurement that the next kernel can change.

Reproducibility

Code and artifacts live in the hexa-lang repository: <https://github.com/dancinlab/hexa-lang>. Kernel: `archive/fires/fusion_attention_flash_2026_05_25/flash_attn.ptx`; structural oracle: `.../structural_oracle.c`; hexa-native closed-form anchor: `.../ratio_check.hexa`; hosts: `tool/fusion_attn_flash_host.c` and `tool/fusion_attn_cublas_baseline.c`; raw verdict: `.verdicts/fusion-attention-flash/F-FUSION-ATTENTION-FLASH.txt`; claim index: `CLAIMS.tape` (group GPU). Build: `cc -O2 structural_oracle.c; hexa build ratio_check.hexa; nvcc -O2 fusion_attn_flash_host.c -lcuda; nvcc -O2 -x cu fusion_attn_cublas_baseline.c -lcublas -lcudart`. Paper build: `make (pdflatex ×3 + bibtex)`.

A Full HBM-traffic derivation

We count, per attention head, the global-memory bytes each decomposition must move, in fp32 (4 bytes/element). The tensors are $Q, K, V, O \in \mathbb{R}^{N \times d}$ and the score matrix $S \in \mathbb{R}^{N \times N}$.

Fused kernel. One launch. It reads Q, K, V ($3Nd$ elements) and writes O (Nd elements). S is held in registers/local across the online loop and never written:

$$H_{\text{tot}}^{\text{fused}} = (3Nd + Nd) \cdot 4 = 4Nd \cdot 4 \text{ bytes}, \quad H_S^{\text{fused}} = 0. \quad (6)$$

cuBLAS baseline. Three launches.

- GEMM-1 ($S = QK^\top/\sqrt{d}$): reads Q, K ($2Nd$), writes S (N^2).
- softmax: reads S (N^2), writes S (N^2).
- GEMM-2 ($O = \text{softmax}(S)V$): reads S (N^2), reads V (Nd), writes O (Nd).

The S round-trip is four passes of N^2 elements:

$$H_S^{\text{base}} = (1 + 1 + 1 + 1) N^2 \cdot 4 = 4N^2 \cdot 4 \text{ bytes}. \quad (7)$$

Tensor traffic is Q, K ($2Nd$) + V (Nd) + O (Nd) plus the GEMM-1 O -less read pattern, totalling $5Nd$ elements of $Q/K/V/O$ movement, so

$$H_{\text{tot}}^{\text{base}} = (5Nd + 4N^2) \cdot 4 \text{ bytes}. \quad (8)$$

The ratio is therefore $R(N, d) = (5Nd + 4N^2)/(4Nd) = 1.25 + N/d$ as in Eq. (5), dominated by the N/d term once $N \gg d$. At $d=64$: $R(512) = 9.25$, $R(1024) = 17.25$, $R(2048) = 33.25$, $R(4096) = 65.25$. The S -round-trip term alone is $4N^2/4Nd = N/d$ times the entire fused tensor traffic — i.e. at $N=2048, d=64$ the baseline moves the score matrix $32\times$ over, while the fused kernel moves it zero times.

B Full silicon measurement (verbatim)

Table 2 reproduces the per-shape silicon numbers verbatim from the verdict file: median wall, mean, standard deviation (% of mean), and the numeric rel-error against the $f64$ CPU reference. The standard deviations are $\leq 0.9\%$ of the mean throughout, so the $1.4\text{--}1.5\times$ wall gap is far outside measurement noise.

Table 2: Verbatim silicon measurement, RTX 5070 (sm_120, driver 580), 20 warmup + 200 timed launches per cell, CUDA-event timing.

N ($d=64$)	fused med (ms)	base med (ms)	fused/base	std (%)	numeric
512	0.415104	0.281696	1.473	≤ 0.90	PASS rel 1.2×10^{-6}
1024	0.822592	0.559360	1.470	≤ 0.51	PASS rel 9.5×10^{-7}
2048	1.639840	1.143040	1.435	≤ 0.32	PASS rel 2.5×10^{-6}
4096	3.788256	2.492288	1.520	≤ 0.18	PASS rel 4.3×10^{-6}

C Structural oracle output (verbatim)

```
F-FUSION-ATTENTION-FLASH STRUCTURAL ORACLE (deterministic, no GPU)
clause (a): fused 1 launch + O(N*d) HBM vs baseline 3 launches + O(N^2) S
shape fused_L base_L fused_Shbm base_Shbm total_ratio
N512/d64 1 3 0 4194304 9.250 x
N1024/d64 1 3 0 16777216 17.250 x
N2048/d64 1 3 0 67108864 33.250 x
N4096/d64 1 3 0 268435456 65.250 x
N2048/d128 1 3 0 67108864 17.250 x
```

```
Closed-form: launches fused=1 vs baseline=3 (3x fewer).  
Closed-form: S-matrix HBM round-trip fused=0 vs baseline= $4 \cdot N^2 \cdot 4$  B ( $O(N^2)$ ).  
At N=2048,d=64: baseline S round-trip = 67108864 B (64.0 MiB); fused S = 0 B.  
F-FUSION-ATTENTION-FLASH (structural clause a): PASS
```

References

- [1] Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. FlashAttention: Fast and memory-efficient exact attention with IO-awareness. *Advances in Neural Information Processing Systems (NeurIPS)*, 2022. URL <https://arxiv.org/abs/2205.14135>.
- [2] Maxim Milakov and Natalia Gimelshein. Online normalizer calculation for softmax. *arXiv preprint*, 2018. URL <https://arxiv.org/abs/1805.02867>.
- [3] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in Neural Information Processing Systems (NeurIPS)*, 2017. URL <https://arxiv.org/abs/1706.03762>.